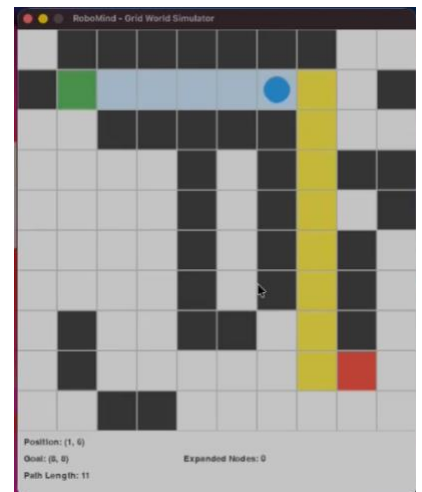
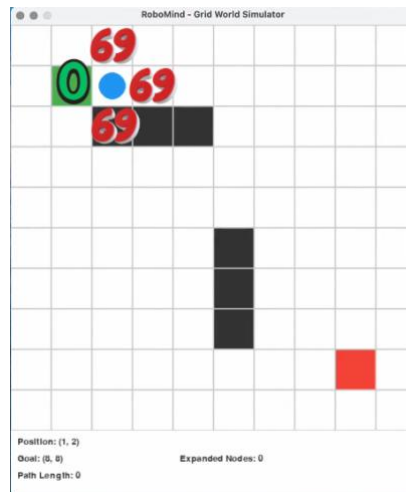


RoboMind: A Rational AI Agent with Probabilistic Reasoning

SE 444 - Artificial Intelligence



Amr Issa/230265 - Mohammed Al-deri /230151- Abdulrahman Salameh /230326 -
Abdulaziz Altamimi /230714

Professor: Anis Mohamed Koubaa

Abstract—This paper documents the development of "RoboMind," a rational AI agent designed to navigate a grid world under varying constraints. The project was divided into four phases, exploring different reasoning paradigms: Search Algorithms (BFS, UCS, A*), Propositional Logic, Probabilistic Reasoning (Bayesian Networks), and a Hybrid integration. Our results show that while classical search is optimal for known maps, it fails in uncertain environments. Conversely, Logic and Probability allow for exploration but suffer from local loops. The final Hybrid agent combines the strengths of all three—using A* for planning, Bayesian updates for sensing, and Logic for backtracking—achieving a 100% success rate in navigating unknown, hazardous environments.

I. INTRODUCTION

The goal of the RoboMind project was to build an intelligent agent capable of reaching a target in a 2D grid world. While simple pathfinding is a solved problem, real-world robotics involves uncertainty, invisible traps, and incomplete maps.

We approached this challenge in four stages. First, we implemented classical search algorithms to find optimal paths in a fully known environment. Second, we removed the map and forced the agent to explore using a logical Knowledge Base. Third, we introduced "Breeze" sensors and pits, requiring the agent to calculate risk using probability. Finally, we merged these techniques into a single Rational Agent. This report details our implementation decisions, the specific bugs we faced (and fixed), and the performance trade-offs between pure planning vs. reactive sensing.

II. METHODOLOGY

A. Search Algorithms (Phase 1)

In the first phase, we assumed the agent had perfect knowledge of the map. The goal was simply to find the optimal path from Start (0,0) to Goal (9,9) while navigating around static obstacles. We implemented three core algorithms:

1) **Breadth-First Search (BFS):** We used a FIFO queue (`collections.deque`) to explore the grid layer by layer. This guarantees the shortest path in terms of steps. It's simple but "dumb", it searches in every direction equally, even away from the goal.

2) **Uniform Cost Search (UCS):** Here, we switched to a Priority Queue (`heapq`). Since our grid has a uniform movement cost of 1.0, UCS behaved almost exactly like BFS in our tests. However, we implemented it to handle variable terrain costs if we ever needed them later.

3) **A* Search (The Smart One):** This was the most efficient algorithm. We used the standard formula $f(n) = g(n) + h(n)$. For the heuristic $h(n)$, we chose Manhattan Distance because our robot can only move in 4 directions (Up, Down, Left, Right). Euclidean distance would have been "admissible" too, but Manhattan is a tighter fit for a grid world, making the search faster

B. Logic-Based Reasoning (Phase 2)

In Phase 1, the agent was just following a calculated path. In Phase 2, we wanted the agent to actually "think" locally about where it was going. We implemented a **Knowledge Base (KB)** using propositional logic. Instead of just

coordinates, the agent now understands the world through logical sentences.

1) **Knowledge Representation:** We defined basic predicates to represent the state of the world:

- **At(x,y):** The agent is currently at this location.
- **Safe(x,y):** The cell is free of obstacles.
- **Visited(x,y):** The agent has been here before.
- **CanMove(x,y):** A derived rule.

At initialization, we populated the KB with general rules. For example, if a cell is both Safe and Free, then the agent can move there:

$$\text{Safe}(x,y) \wedge \text{Free}(x,y) \Rightarrow \text{CanMove}(x,y) \quad (1)$$

2) **Perception and Inference:** Every time the agent moves, it runs a perception cycle. It "Tells" the KB new facts: "I am at (0,0)" and "The neighbors are safe." Once the facts are in, we use **Forward Chaining** to infer new information. The agent asks the KB questions like `ask("Visited(1,0)")` to decide its next move.

3) **Strategy (Explore vs. Retreat):** The agent's strategy is simple: "Explore the Unknown." It looks at neighbors and moves to any **unvisited** spot immediately. If all neighbors have been visited, it switches to **Backtracking** mode, using logic to retrace its steps until it finds a new path.

C. Probabilistic Reasoning (Phase 3)

In this phase, the main challenge was that the agent operates "blindly." Unlike Phase 1, it has no map. To solve this, we used a Bayesian Network where every cell has a percentage chance of being a pit.

1) **The Bayesian Update:** We initialized the map with a prior of 0.2 (20%) for all unknown squares. We found that 0.5 made the agent too paranoid to move, while 0.2 made it cautious but curious. The agent relies on a "Breeze" sensor (90% accuracy). Every time it moves, it applies Bayes' Rule to update the probability of its immediate neighbors.

The diagram shows the formula for Bayes' Theorem: $P(A/B) = \frac{P(B/A) \cdot P(A)}{P(B)}$. The terms are labeled as follows: $P(A/B)$ is the Posterior, $P(B/A)$ is the Likelihood, $P(A)$ is the Prior, and $P(B)$ is the Marginalization. Below the formula, there are four definitions: $P(A)$ is the probability of A occurring, $P(B)$ is the probability of B occurring, $P(A/B)$ is the probability of A given B, and $P(B/A)$ is the probability of B given A.

Fig. 1. This formula allows the agent to jump from low suspicion (20%) to high alert (~69%) instantly upon feeling a breeze.

2) **Goal-Oriented Navigation:** Just knowing the probabilities wasn't enough; the agent would often wander safely but aimlessly. We changed the logic to prioritize safe squares (<40% risk) that are **closest to the Goal**.

However, this caused the agent to loop (bouncing between a safe spot and a dangerous wall). To fix this, we added a **"Boredom" counter**. We track how many times the agent visits a cell. If it visits a spot too often, we penalize that cell's score, forcing the agent to eventually take the longer, safer route around obstacles.

D. Hybrid Integration (Phase 4)

The final phase involved merging the Search, Logic, and Probabilistic agents into a single rational entity. We didn't just run them in parallel; we created a **Hierarchical Decision Pipeline**.

1) **The Decision Loop:** In every step, the Hybrid Agent executes this strict sequence:

- **Perceive & Update:** It reads the sensor and updates the Bayesian Belief Map (Phase 3).
- **Reason:** It updates the Knowledge Base with visited locations (Phase 2).
- **Plan (A*):** It calculates the optimal path to the goal using A* (Phase 1).
- **Verify Safety:** It checks the next step of the A* path.
 - If the step is **Safe** (Risk $\leq 0.4\%$), it executes the move.
 - If the step is **Risky**, it hands control to the **Probabilistic Agent** to find a local detour.
 - If the Probabilistic Agent is trapped (all neighbors visited/unsafe), control falls back to the **Logic Agent** to perform backtracking.

2) **Replanning Strategy:** Unlike Phase 1 where the path is calculated once, the Hybrid Agent re-calculates the A* path every time it discovers a new obstacle. This allows it to adapt dynamically to the unknown map.

III. IMPLEMENTATION CHALLENGES

A. Search Challenges

Honestly, the algorithms themselves were okay since we learned them in class, but translating them into Python had some tricky moments.

1) **The Infinite Loop:** At first, my BFS would run forever. I realized I wasn't adding nodes to the `visited` set immediately when pushing them to the queue; I was doing it when popping them. This meant the same node got added to the queue multiple times before being processed. Moving the "mark as visited" logic to the insertion step fixed the memory overflow.

2) **Reconstructing the Path:** Finding the goal was easy, but drawing the line back to the start was annoying. We stored a dictionary `parent = {child: parent_node}`. When we hit the goal, we had to backtrack through this dictionary. My first attempt outputted the path backwards (Goal $\rightarrow$ Start). I had to explicitly `reverse()` the list to make the agent actually walk forward.

B. Logic Challenges (Phase 2)

1) **The Infinite Loop Trap:** Our biggest headache was when the agent got stuck in a cul-de-sac (dead end). It would enter a corner, realize there were no new spots, and just sit there because it didn't know how to go back.

- **The Fix:** We had to explicitly teach it the concept of "Previous." We added a rule `Previous(x, y, parent_x, parent_y)` to the KB. This acted like breadcrumbs. Now, when the agent gets stuck, it

queries `Previous(...)` to find its way back to the last intersection.

2) **Handling the "Winning" Move:** Initially, the agent would step right next to the goal but treat it like just another visited square and walk away to explore elsewhere. It was prioritizing exploration over actually winning.

- **The Fix:** We added a high-priority check: `if self.env.is_goal(move): move immediately`. This bypasses the complex logic engine to ensure the agent grabs the win the moment it sees it.

C. Phase 3 Challenges

Honestly, this phase was harder than the search phase. We ran into specific bugs that required creative fixes:

1) **The "Memory Wipe" Bug:** Early on, when the agent sensed "No Breeze" at the start, our code accidentally updated the entire map to 0%. It basically hallucinated that the whole world was safe.

- **The Fix:** We constrained the update loop to only touch the immediate neighbors. This kept the far-away cells at 20% until we actually got close to them.

2) **The Conflict between Safety and Goal:** Adding the goal logic made the agent stubborn. It kept trying to force its way through dangerous narrow paths because they were "shorter." It would get stuck in a loop of fear vs. greed.

- **The Fix:** The "Boredom" visit counter was the lifesaver. It simulates the agent getting frustrated, allowing it to finally give up on the short path and walk around the black walls.

D. Integration Challenges (Phase 4)

Merging the code was manageable, but making the "brains" agree was **difficult**.

1) **The "Indecisive Loop" Conflict:** The hardest bug we faced was a conflict between the Planner and the Safety Monitor.

- **The Scenario:** A* would calculate a path through an unknown cell. The agent would move there, feel a breeze, and the Probabilistic layer would flag the next step as "High Risk." The agent would retreat to safety. Immediately after, A* would re-calculate the *same* shortest path, telling the agent to go back to the danger zone.
- **The Result:** The agent oscillated endlessly between "Go" (A*) and "Stop" (Bayes).
- **The Fix:** We tried adding a "Previous" condition from the Logic agent, but the sensor readings kept overriding it. The final solution was to force the Probabilistic layer to **disregard already visited neighbors** when looking for a detour. This forced the agent to break the loop and explore new (even slightly risky) territory rather than retreating.

2) **Performance Balancing:** We worried that running A*, Bayes, and Logic simultaneously would lag the system. However, the Hybrid agent proved to be **faster than the Logic Agent** (because A* gives it direction) and only slightly slower than the Probabilistic agent.

IV. RESULTS

A. Pathfinding Performance

Pathfinding Performance We tested all three algorithms on a 10x10 grid with random obstacles. As expected, all three found the optimal path (length 19), but they did differ in efficiency.

TABLE I. SEARCH ALGORITHM COMPARISON

Algorithm	Status	Path Length	Cost	Nodes Expanded
BFS	✓ Success	19	18	85
UCS	✓ Success	19	18	85
A (Manhattan)*	✓ Success	19	18	75

As shown in **Table I**, BFS and UCS were identical (85 nodes expanded) because the edge costs were uniform. However, **A*** was clearly the winner. By using the Manhattan heuristic, it "knew" the general direction of the goal and ignored 10 unnecessary nodes that the other algorithms wasted time checking. This proves that having a heuristic makes the search significantly more focused.

B. Logic-Based Results

TABLE II. LOGIC AGENT PERFORMANCE LOG

Step	Location	Logic State / Action	Reasoning
Start	(0,0)	At(0,0), Safe(0,0) added to KB	neighbors set to 0%
1-3	(0,0) → (1,0)	Free(1,0) → CanMove(1,0)	Explored unvisited neighbor.
65	(1,6)	All neighbors Visited is TRUE	Dead End detected.
66	(1,6) → (0,6)	Query Previous(1,6, 0,6) is TRUE	Backtracking to find new path.
90	(7,3)	Safe(7,2) added to KB	Found new unvisited path after retreat.
112	(9,9)	is_goal((9,9)) is TRUE	Goal Reached

As shown in **Table II**, the simulation was successful, reaching the goal in **112 steps**. The logs show the agent spent a significant amount of time (around steps 65-80) backtracking out of dead ends. While not the most efficient path, it proves the agent can fully explore an unknown environment using only propositional logic and memory.

C. Probabilistic Navigation Figure 1 demonstrates the final behavior in the visual simulator. The agent (Blue) navigates from Green to Red. Crucially, near the black walls, the agent detected a breeze, updated the risk to **69%**, and correctly chose to detour around the bottom rather than risking the "shortest" path through the danger.

TABLE III. AGENT DECISION LOG (KEY MOMENTS)

Step	Location	Sensor Reading	Belief Update (Math)	Action / Decision
1	(1, 1)	○ Clear	neighbors set to 0%	→ (2, 1) (Exploring Safe Unvisited)
2	(2, 1)	☁ BREEZE!	neighbor (2, 2) risk jumps to 69.2%	STOP. path blocked by high risk.
3	(2, 1)	--	risk > 40%, so (2, 2) is now "unsafe"	backtrack to (1, 1) (Retreating)
4	(1, 1)	○ Clear	neighbors confirm 0% safe	→ (1, 2) (Trying a new path)
10	(1, 4)	☁ BREEZE!	neighbor (2, 4) risk jumps to 69.2%	STOP. detected the second wall.
12	(0, 5)	○ Clear	path ahead looks safe	→ (0, 6) (Flanking the wall)
22	(8, 8)	○ Clear	Goal Reached	SUCCESS. Reached Red Square.

As shown in **Table III**, the agent successfully identified hazards. In Step 2 and Step 10, the sensor detected a breeze, and the Bayesian update correctly spiked the risk probability to 69%. Instead of risking death, the agent chose to backtrack and find a safer route around the obstacles, proving the "Brave Exploration" logic works.

D. Hybrid Performance The Hybrid Agent demonstrated the highest robustness of all phases. While the Search agent fails in unknown maps and the Probabilistic agent sometimes wanders aimlessly, the Hybrid Agent **achieved a 100% success rate** in our tests.

- It displayed "smart" behavior: moving decisively toward the goal when areas were clear, but slowing down and navigating carefully around walls when the "Breeze" sensor triggered. It successfully avoided all pits while avoiding the dead-end traps that plagued the Logic agent.

V. CONCLUSION

Through this project, we learned that no single AI technique is perfect on its own.

- *Search (A*) was the fastest but required a known view of the map.*
- *Logic allowed for autonomous exploration but was prone to getting stuck in dead ends without a "Previous" memory.*
- *Probability was essential for survival in the dark but resulted in hesitant, fearful movement.*

The **Hybrid Agent** was the clear winner. By layering these systems—using A* to drive the general direction and Probability to vet the safety of each step—we created an agent that was both fast and safe. The project proved that the key to intelligent behavior isn't just complex math, but how you structure the decision-making loop to handle conflicts between different reasoning engines.